Let's Encrypt Hermetic HTTPS — Deep Research (Caddy + Pebble + challtestsrv, DNS-01)

Revision: 2 **Last modified:** 2026-07-01T10:06:36Z **Authority:** Helix Constitution §11.4.150 (deep multi-angle research before workstream commitment) + §11.4.99 (latest-source cross-reference) + §11.4.6 (no-guessing — every claim cited or marked UNCONFIRMED). **Scope:** de-risk the hermetic (offline) issuance + auto-renewal phases (Phase 3 build-out, Phase 5 rotation test) of the Let's Encrypt HTTPS workstream in `helix_proxy`. **Locked operator decisions (inputs, not under research):** TLS client = **Caddy auto-HTTPS** (Caddy is BOTH the in-process ACME client AND the TLS terminator; renewal is Caddy's in-process loop — NO cron / CI / systemd timer / hooks, per §11.4.156); ACME challenge = **DNS-01**; rollout = **hermetic (Pebble) + LE-staging first**, real domain deferred. **Consumes:** `tests/letsencrypt/cert_analyzer.sh` (`cert_chain_roots_in` / `cert_not_expired` / `cert_san_matches` / `cert_days_remaining` / `cert_renewal_due`) — the client-and-challenge-AGNOSTIC PEM analyzer that scores the RESULTING certificate.

> **Method note (§11.4.6 / §11.4.99):** all facts below are cited to primary sources (project READMEs / official Caddy docs / maintainer statements) fetched **2026-07-01**. Where a source is silent or I could not confirm from a primary source, the claim is marked **UNCONFIRMED** with the concrete method to obtain the fact at runtime — never guessed.

Q1 — Pebble as a local hermetic ACME server

What it is. Pebble (`letsencrypt/pebble`) is a miniature RFC-8555 ACME test CA — "a small RFC 8555 ACME test server not suited for a production certificate authority." It is the intended offline stand-in for Boulder / Let's Encrypt.

How to run it (three forms, all cited from the README):

```
# (a) container image (recommended for hermetic use)
docker run -p 14000:14000 -p 15000:15000 -e "PEBBLE_VA_NOSLEEP=1"
        \
  ghcr.io/letsencrypt/pebble:latest

# (b) docker-compose (the repo ships a compose file)
docker-compose up

# (c) binary
pebble -config ./test/config/pebble-config.json
```

Ports / endpoints (README):

Purpose	Value
ACME **directory URL**	`https://localhost:14000/dir`
ACME port	`14000`
Management interface (HTTPS)	`https://localhost:15000`
Issuance **root** (fetch at runtime)	`GET https://localhost:15000/roots/0`
Issuance **intermediate** (fetch at runtime)	`GET https://localhost:15000/intermediates/0`
Alt root retrieval (also seen)	`GET https://localhost:14000/root`
Default HTTP-01 validation port	`5002`
Default TLS-ALPN-01 port	`5001`

Its test CA — TWO distinct CAs (do not conflate; this is the #1 gotcha, see Q6):

- **`test/certs/pebble.minica.pem`** signs Pebble's **own HTTPS endpoints** (the `:14000` directory and `:15000` management interface). A client (Caddy, curl) must trust THIS to *talk to* Pebble over HTTPS. Files: `test/certs/pebble.minica.pem`, `test/certs/pebble.minica.key.pem`, `test/certs/localhost/` (the end-entity cert for the Pebble HTTPS server). This CA is **static in the repo** and its private key is **public** — README: "Do not add the `pebble.minica.pem` CA to the system-wide trust store."
- The **issuance root/intermediate** returned by `/roots/0` + `/intermediates/0` sign the certificates Pebble **issues to ACME clients**. This is what the test asserts the resulting leaf chains to (Q5).

Caveats — Pebble deliberately randomizes (README):

- Issuance CA is **regenerated on every launch** — "Pebble's root certificate is regenerated on every launch… Pebble does not support non-volatile storage… and will randomize keys/

certificates used for issuance" (issue #152 / README). **Consequence:** a *static* expected-CA fixture is INVALID across restarts — the test MUST fetch `/roots/0` (+ `/intermediates/0`) dynamically each run (drives Q5 design).
- Validation Authority sleeps a random **0–15 s** between attempts unless `PEBBLE_VA_NOSLEEP=1`.
- Rejects **5 %** of valid nonces by default (`PEBBLE_WFE_NONCEREJECT`, set `=0` to disable).
- Reuses authorizations ~50 % of the time (`PEBBLE_AUTHZREUSE`).
- `-strict false` recommended for test stability.

Key env vars (README): `PEBBLE_VA_NOSLEEP=1`, `PEBBLE_VA_ALWAYS_VALID=1` (skips actual challenge validation — see Q3 fidelity note), `PEBBLE_VA_SLEEPTIME`, `PEBBLE_WFE_NONCEREJECT=0`, `PEBBLE_ALTERNATE_ROOTS`, `PEBBLE_CHAIN_LENGTH` (chain depth, min 1).
DNS wiring flag: `pebble -config … -dnsserver <host>:8053` points Pebble's VA at an external DNS server (challtestsrv) instead of the system resolver — mandatory for hermetic DNS-01.

UNCONFIRMED: the exact validity period (in days) of a Pebble-*issued* leaf. No primary source consulted stated a number, and I found no config knob for it. Method to obtain at runtime: `openssl x509 -in <leaf.pem> -noout -startdate -enddate`. (This does not block Phase 5 — the force-renewal lever in Q4 is validity-period-independent.)

Sources: pebble README, pebble repo, test/certs README, issue #152 "get hold of root certificate", LE blog "How Pebble Supports ACME Client Developers" — accessed 2026-07-01.

Q2 — pebble-challtestsrv: serving DNS-01 responses hermetically

What it is. `pebble-challtestsrv` (bundled in `letsencrypt/pebble` under `cmd/pebble-challtestsrv`; library form `letsencrypt/challtestsrv`) is a TEST-ONLY mock challenge + DNS server: it runs HTTP-01, HTTPS HTTP-01, DNS-01 and TLS-ALPN-01 responders plus a **mock DNS server** answering `A/AAAA/TXT/CNAME/CAA`, all mutated live via an **unauthenticated HTTP management API**. README: "trivially insecure, offering no authentication whatsoever. Only use … in a controlled test environment."

Command-line flags + DEFAULT ports (cmd README):

Flag	Default	Role
`-dnsserver`	`:8053`	**mock DNS server** (UDP+TCP) — Pebble's VA and Caddy's self-check point here

Flag	Default	Role
`-management`	`:8055`	**HTTP management API** (set/clear records)
`-http01`	`:5002`	HTTP-01 responder (unused for DNS-01)
`-https01`	`:5003`	HTTPS HTTP-01 responder (unused)
`-tlsalpn01`	`:5001`	TLS-ALPN-01 responder (unused)
`-defaultIPv4`	`127.0.0.1`	default A answer
`-defaultIPv6`	`::1`	default AAAA answer (see IPv6 gotcha, Q6)

Publishing / clearing a DNS-01 TXT record (management API on :8055):

```
# publish the challenge TXT (NOTE the REQUIRED trailing '.' on
        host)
curl -d '{"host":"_acme-challenge.proxy.test.",
        "value":"<base64url-challenge-digest>"}' \
  http://localhost:8055/set-txt

# clear it after validation
curl -d '{"host":"_acme-challenge.proxy.test."}' \
  http://localhost:8055/clear-txt
```

Mock A / defaults (used to make the FQDN resolve inside the podman net):

```
curl -d '{"host":"proxy.test.", "addresses":["10.89.0.20"]}'
        http://localhost:8055/add-a
curl -d '{"ip":"10.89.0.20"}'  http://localhost:8055/set-default-
        ipv4
curl -X POST -d '{"ip":""}'   http://localhost:8055/set-default-
        ipv6   # blank AAAA — see Q6
```

How the CA resolves it. The mock DNS server listens on `:8053` and answers `A/AAAA/TXT/CAA`. When Pebble's VA validates DNS-01 it issues a `TXT _acme-challenge.<domain>` query to whatever `-dnsserver` points at; challtestsrv returns the value previously POSTed to `/set-txt`. This closes the loop **with no real DNS provider and no real domain**.

Sources: pebble-challtestsrv cmd README, challtestsrv repo, challtestsrv README, Ubuntu manpage — accessed 2026-07-01.

Q3 — Caddy DNS-01: does it need a custom (xcaddy) build? Can challtestsrv be used without a provider plugin?

Fact 1 — mainline Caddy needs a DNS provider MODULE for ANY DNS-01. The official Caddy binary/image ships **no** DNS provider modules. The `tls` directive's `dns` subdirective requires a module "which must be plugged in from one of the `caddy-dns` repositories," compiled at build time with **xcaddy**:

```
# custom image = stock caddy builder + the DNS provider module
FROM caddy:builder AS build
RUN xcaddy build --with github.com/caddy-dns/<provider>
FROM caddy:latest
COPY --from=build /usr/bin/caddy /usr/bin/caddy
```

Fact 2 — there is NO off-the-shelf `caddy-dns` provider for challtestsrv. challtestsrv exposes its own HTTP `set-txt`/`clear-txt` API (Q2); stock Caddy cannot drive it, because Caddy's DNS-01 solver writes the TXT through a **libdns provider** (certmagic → acmez → libdns), not through challtestsrv's bespoke API. `PEBBLE_VA_ALWAYS_VALID` / `dns_challenge_override_domain` do **not** remove the provider requirement — Caddy still needs a provider to *attempt* DNS-01. So challtestsrv **cannot** be used by stock Caddy without wiring. This is the honest crux of Q3.

Therefore, three hermetic options (pick one in Phase 3):

- **Option A — tiny custom libdns provider → challtestsrv (fully hermetic, minimal SERVICES, ~60 LOC).** Write a small Caddy/libdns module (`dns.providers.challtestsrv`) implementing `libdns.RecordAppender` + `RecordDeleter` that POSTs to challtestsrv `:8055/set-txt` and `/clear-txt`. Compile via `xcaddy build --with <that-module>`. This genuinely exercises Caddy's DNS-01 code path end-to-end against the SAME challtestsrv the CA queries. Cost: one small owned Go module to maintain (extend-don't-reimplement, §11.4.74 — could live in a `caddy-dns/challtestsrv`-style upstream).

- **Option B — acme-dns server + existing `caddy-dns/acmedns` (zero custom code, more SERVICES).** Run `joohoi/acme-dns` as the authoritative DNS + build Caddy `--with github.com/caddy-dns/acmedns`. Caddyfile:

  ```
  tls {
    dns acmedns {
      username <u>
      password <p>
      subdomain <subdomain>
      server_url http://acme-dns:8080
  ```

```
  }
}
```

Pebble's `-dnsserver` points at acme-dns (`:53`); Caddy writes TXT via acme-dns's HTTP API. **challtestsrv is not used in this variant.** Cost: acme-dns has its own SQLite + a one-time account-registration step (a §11.4.98(B)-permissible bootstrap, outside the test), and requires a `_acme-challenge` CNAME → acme-dns subdomain (acme-dns serves it, so still hermetic).

- **Option C — `PEBBLE_VA_ALWAYS_VALID=1` (NOT a DNS-01 proof — reduced fidelity).** Skips challenge validation entirely, so NO provider/challtestsrv is needed and issuance+chain+renewal plumbing can be smoke-tested. **But it does not exercise DNS-01 at all** — using it as *the* DNS-01 proof would be a §11.4 PASS-bluff (green while the DNS-01 path is unproven). Acceptable only as a first plumbing smoke, explicitly labelled, never as the Phase-5 evidence.

Recommendation: Option A for the tightest hermetic footprint (Pebble + challtestsrv + custom-Caddy, three services, one shared DNS mock for both the CA and Caddy's self-check) — OR **Option B** if the operator prefers zero owned Go over one extra service. Both are genuinely hermetic and genuinely exercise DNS-01. Option C is a labelled smoke only.

Sources: Caddy `tls` directive, How to use DNS provider modules in Caddy 2, xcaddy, caddy-dns/acmedns, certmagic (libdns DNS01Solver), caddy-dns/cloudflare Dockerfile example — accessed 2026-07-01.

Q4 — Caddy auto-renewal: trigger window, forcing a near-expiry renewal, zero-downtime reload, observability

Renewal window (default & formula). Caddy renews in-process when the remaining lifetime drops to the `renewal_window_ratio`. Default **`0.3333`** → renew when ≤ 1/3 of the lifetime remains (30 d on a 90 d cert). Exact certmagic formula:

```
renewAt = notBefore + (notAfter - notBefore) * (1 -
renewal_window_ratio)
```

Both a **global** option and a per-site `tls { renewal_window_ratio <r> }` subdirective. **ACME ARI** (Automated Renewal Information) may override this ratio if the CA advertises it (the CA dictates the window) — a possible confounder for a deterministic rotation test (see Q6).

Requested cert lifetime. Global `cert_lifetime <duration>` — "validity period to ask the CA to issue a certificate for. Default `0` (CA chooses, usually 90 days)." (Whether Pebble honors a requested `notAfter` is **UNCONFIRMED** — not needed for the force-renew lever below.)

Forcing a near-expiry renewal (rotation test) — two authoritative methods:

- **(preferred, deterministic) `renewal_window_ratio 1` + reload.** Per Caddy author Matt Holt: "Caddy will immediately renew the certificate regardless of scan time, because it always checks when the config is first loaded." Setting the ratio to `1` makes `renewAt ≈ notBefore`, so the current cert is already "due" and Caddy renews on the next config load. Revert afterwards (rate-limit hygiene — moot against Pebble).
- **(alternative) delete the cert from storage + reload.** Remove `<storage>/certificates/<acme-ca-host>-directory/<domain>/<domain>.{crt,key,json}` then `caddy reload`; Caddy re-issues automatically ("the first request against this domain performed the certificate renewal").

Zero-downtime reload / hot-swap. Caddy applies new config with graceful, zero-downtime reload (`caddy reload --config …` or admin API `POST /load`); on renewal it "swaps out the old certificate with the new one … zero downtime" — connections are not dropped across the swap. (Caveat: a config *reload* flushes the in-memory cert cache and reloads from storage — issue #5589 — so distinguish "renewal hot-swap" (no downtime) from "full config reload".)

Observability (how a test SEES a renewal happened):

- **Admin API** on `localhost:2019` (global `admin <addr>`, default `localhost:2019` or `CADDY_ADMIN`) — `GET /config/` to read live config; `POST /load` to reload. No dedicated "force-renew" admin endpoint exists.
- **Structured logs** — Caddy emits a certificate-obtained/renewed log event. **UNCONFIRMED exact string** (docs consulted did not quote it); method to capture: run Caddy with JSON logging and grep the log stream for the certificate-management logger (`tls`/`tls.obtain`) emitting an obtained/renewed entry. Treat the log as corroborating evidence, not the sole proof.
- **The cert itself is the primary evidence** — compare leaf **serial** and **notBefore** before vs after (`openssl x509 -noout -serial -startdate`): a changed serial + advanced notBefore proves a real re-issue (not a cache reload).

Sources: Caddy `tls` directive, Caddy global options, Caddy automatic-https, certmagic, How to force renewal (Matt Holt), cert-cache-flush issue #5589, DeepWiki cert storage & caching — accessed 2026-07-01.

Q5 — Trust/verification: asserting the resulting cert chains to the Pebble test CA

Runtime CA retrieval (mandatory — Pebble regenerates the issuance CA each launch, Q1):

```
# fetch THIS run's issuance chain from Pebble's management
        interface (-k: its own
# endpoint is signed by pebble.minica.pem, NOT the issuance CA —
        see Q6)
curl -sk https://pebble:15000/roots/0         -o pebble_root.pem
curl -sk https://pebble:15000/intermediates/0 -o
        pebble_intermediate.pem
cat pebble_intermediate.pem pebble_root.pem   >
        pebble_ca_bundle.pem
```

Pull the leaf Caddy is actually serving (proves the deployed artifact, not storage):

```
openssl s_client -connect caddy:443 -servername proxy.test </dev/
        null 2>/dev/null \
  | openssl x509 > served_leaf.pem
# (alternatively read <storage>/certificates/<ca>-directory/
        proxy.test/proxy.test.crt)
```

Feed the existing analyzer (`tests/letsencrypt/cert_analyzer.sh`):

```
. tests/letsencrypt/cert_analyzer.sh
cert_chain_roots_in served_leaf.pem pebble_ca_bundle.pem   #
        issued by the Pebble run's CA
cert_not_expired    served_leaf.pem                        #
        inside validity window now
cert_san_matches    served_leaf.pem proxy.test             # SAN
        covers the served host
cert_days_remaining served_leaf.pem                        #
        scalar for reporting
```

Integration note (load-bearing — matches the analyzer's implementation). `cert_chain_roots_in` runs `openssl verify -no_check_time -CAfile <expected_ca_pem> <leaf>`. Pebble issues a **leaf → intermediate → root** chain (`PEBBLE_CHAIN_LENGTH` ≥ 1), so verifying against the **root alone** FAILS ("unable to get local issuer certificate"). Pass the **root+intermediate BUNDLE** as the `expected_ca_pem` (as above) — `openssl verify` treats every cert in `-CAfile` as a trust anchor, so the bundle both supplies the

intermediate and proves the leaf chains into the Pebble run's CA. (Alternative: set `PEBBLE_CHAIN_LENGTH=0` for a direct leaf ← root so a single-root `-CAfile` suffices — but the bundle approach is closer to production chain shape and needs no Pebble reconfiguration.) `-no_check_time` keeps the issuance check orthogonal to `cert_not_expired` (an expired-but-correctly-rooted cert still passes the chain check and fails the expiry check — by design).

Sources: test/certs README, pebble issue #152, Xoxzo "Root certificates generation using Pebble", LE community "cannot get local issuer certificate querying Pebble" — accessed 2026-07-01.

Q6 — Common pitfalls / gotchas (honest negative findings, §11.4.99)

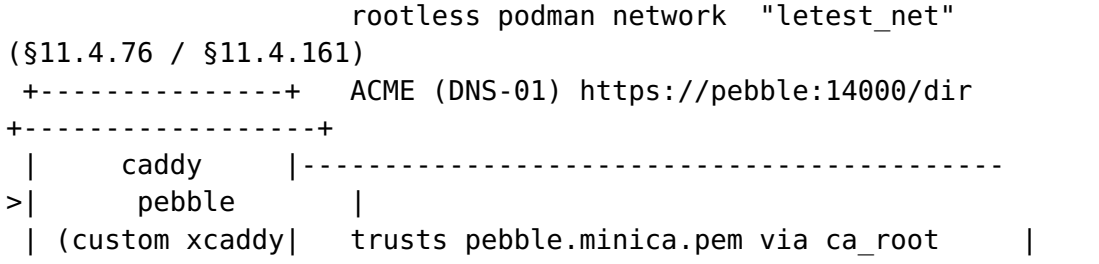
1. **TWO CAs — the #1 conflation trap.** `pebble.minica.pem` signs Pebble's OWN `:14000`/`:15000` HTTPS endpoints → Caddy must trust it (via `acme_ca_root`/`ca_root`) *to talk to Pebble*. The `/roots/0` issuance CA signs what Pebble ISSUES → the test asserts the leaf chains to *that*. Trusting the wrong one = "x509: certificate signed by unknown authority" on the ACME connection, or a chain assertion that never passes.
2. **Pebble regenerates the issuance CA every launch** (Q1) → NEVER pin a static expected-CA fixture; fetch `/roots/0` + `/intermediates/0` at runtime each run. A committed golden CA PEM will pass once then break on the next boot.
3. **DNS resolver wiring is TWO-sided.** (a) Pebble's VA must be pointed at challtestsrv: `pebble … -dnsserver challtestsrv:8053` (Pebble does NOT use system DNS for validation when set). (b) **Caddy's own pre-flight propagation self-check** must ALSO resolve via challtestsrv — set `tls { resolvers challtestsrv:8053 }`, else Caddy queries the system resolver, never sees the TXT, and stalls until `propagation_timeout`. Keep `propagation_delay`/`propagation_timeout` small in-test.
4. **challtestsrv default AAAA `::1` interferes** (confirmed in the field). Its default IPv6 answer can take precedence over your A record; blank it: `curl -X POST -d '{"ip":""}' http://challtestsrv:8055/set-default-ipv6`.
5. **Trailing `.` required** on `set-txt`/`clear-txt`/query host names (`_acme-challenge.proxy.test.`). Missing dot = no match = validation timeout.
6. **Pebble randomization can look like flakiness.** Default 0–15 s VA sleep and 5 % nonce rejection → set `PEBBLE_VA_NOSLEEP=1`, `PEBBLE_WFE_NONCEREJECT=0`, and `-strict false`. acmez/Caddy retries nonces automatically, but tight per-attempt timeouts can still surface it.
7. **Pebble management interface is HTTPS**, signed by `pebble.minica.pem` — curl needs `-k` (or trust minica). Fetching `/roots/0` without `-k`/trust fails.

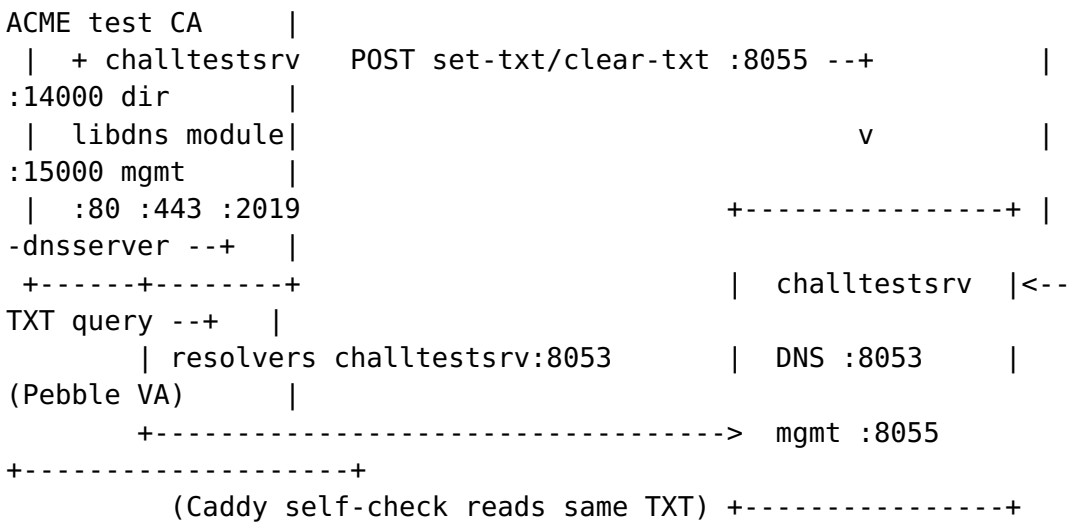
8. **stock Caddy image has no DNS module** (Q3) → a custom xcaddy build is mandatory; do the build via the containers submodule / §11.4.173 containerized build, never an ad-hoc host `docker build`.
9. **Admin API is plaintext HTTP on `:2019`, bound to `localhost` by default.** To reach it from a sibling container/test, set `admin 0.0.0.0:2019` (or `CADDY_ADMIN`) — but treat it as test-only (no auth).
10. **ACME ARI may override `renewal_window_ratio`.** If Pebble advertises `renewalInfo`, Caddy honors the CA-dictated window, which can defeat a ratio-based rotation test. The `renewal_window_ratio 1` + reload lever (Q4) forces renewal at config-load time regardless — prefer it. **UNCONFIRMED** whether this Pebble build serves ARI; verify by watching for a `renewalInfo` request in Pebble logs, and fall back to the delete-storage+reload method if ARI interferes.
11. **Config reload vs renewal hot-swap.** A full `POST /load` flushes the in-memory cert cache and reloads from storage (issue #5589); the *renewal* swap is the zero-downtime one. For the zero-downtime assertion, force renewal (ratio=1 or delete+reload) and probe availability across the swap — don't conflate a cache-flush blip with a renewal.
12. **Clock/validity.** Container clock must be sane (Pebble issues time-bounded certs). `cert_lifetime` asks the CA for a lifetime but Pebble's honoring is **UNCONFIRMED**; the force-renew lever is validity-independent so this does not block Phase 5.

Sources: LE community "pebble-challtestsrv in local docker", pebble README, pebble-challtestsrv cmd README, Caddy `tls` directive, Caddy global options, caddy issue #5589 cert cache flush — accessed 2026-07-01.

Recommended hermetic architecture (Phase 3)

Chosen path: Option A — Pebble + pebble-challtestsrv + custom-built Caddy (custom libdns → challtestsrv), three services, fully offline, DNS-01 genuinely exercised. (Option B — acme-dns + `caddy-dns/acmedns` — is the drop-in fallback if the operator prefers no owned Go over one extra service; wiring is identical except challtestsrv→acme-dns and no custom module.)





Service declarations — via the containers submodule (`submodules/containers`, `pkg/compose` + `pkg/boot`), NOT ad-hoc podman (repo boots ONLY through the submodule; §11.4.76). Declare three `compose.HelixService` entries (real API surface confirmed in `submodules/containers/pkg/compose/helix_project.go`: `HelixService`, `HelixHealthCheck`, `HelixResourceLimits`, `PortMapping`; construct with `compose.NewHelixComposeProject(name, services)`), boot via `boot.BootManager.BootAll` / `orchestrator.Up`, and health-check via `pkg/health`:

Service	Image / build	Ports (host:container)	Key env / command
pebble	ghcr.io/letsencrypt/pebble:latest	14000:14000, 15000:15000	PEBBLE_VA_NOSLEEP=1, PEBBLE_WFE_NONCEREJECT=0; cmd pebble -config /test/config/pebble-config.json -dnsserver challtestsrv:8053 -strict false
challtestsrv	ghcr.io/letsencrypt/pebble-challtestsrv:latest	8053:8053/udp, 8053:8053/tcp, 8055:8055	cmd pebble-challtestsrv -defaultIPv6 "" (blank AAAA, gotcha #4)
caddy	**custom xcaddy image** (xcaddy build --with <challtestsrv-libdns-module>), built via containers submodule (§11.4.173)	80:80, 443:443, 2019:2019	mount Caddyfile + pebble.minica.pem; admin 0.0.0.0:2019

Caddyfile (Phase-3 hermetic, DNS-01, Option A):

```
{
    admin 0.0.0.0:2019
    acme_ca      https://pebble:14000/dir      # Pebble ACME
directory (global)
    acme_ca_root /etc/caddy/pebble.minica.pem   # trust Pebble's
OWN https endpoint
    email        hermetic@proxy.test
    # (Phase-5 rotation flips renewal_window_ratio to 1 — see
below)
}

proxy.test {
    tls {
        dns challtestsrv http://challtestsrv:8055   # custom
libdns -> set-txt/clear-txt
        resolvers challtestsrv:8053                  # Caddy self-
check reads the same mock DNS
        propagation_delay   2s
        propagation_timeout 30s
    }
    respond "hermetic-ok"
}
```

Bootstrap ordering: boot `challtestsrv` + `pebble` first (health-gated), POST the `add-a`/`set-default-ipv4` + `set-default-ipv6 ""` records to challtestsrv `:8055`, then boot `caddy` (its first request triggers issuance). Pebble's per-run `/roots/0` + `/intermediates/0` are fetched at assertion time (Q5), never pinned.

LE-staging step (after hermetic passes): same Caddyfile with `acme_ca https://acme-staging-v02.api.letsencrypt.org/directory`, drop `acme_ca_root`/`resolvers`, and swap the DNS provider to the real one — deferred until a real domain is available.

Renewal/rotation test approach (Phase 5)

Goal: prove Caddy's IN-PROCESS renewal loop genuinely re-issues a new cert, that the new cert still chains to the (same-run) Pebble CA and is valid, and that the swap is zero-downtime — all hermetic, autonomous, re-runnable (§11.4.98), with captured physical evidence (§11.4.5/§11.4.69/§11.4.107).

Force lever (deterministic, authoritative — Q4): set `renewal_window_ratio 1` (global) and `caddy reload` / `POST :2019/load` => Caddy renews immediately at config-load ("it always checks

when the config is first loaded" — Matt Holt). Fallback if ARI interferes (gotcha #10): delete `<storage>/certificates/<ca>-directory/proxy.test/proxy.test.{crt,key,json}` + reload.

Procedure (RED-on-broken -> GREEN, §11.4.115; extend-to-all-cases, §11.4.146):

1. **Baseline** — after Phase-3 issuance, capture leaf #1: `openssl s_client -connect caddy:443 -servername proxy.test | openssl x509 > leaf_1.pem`; record `serial_1`, `notBefore_1` (`openssl x509 -noout -serial -startdate`).
2. **Availability probe on** — start a tight `curl https://caddy/ --resolve proxy.test:443:<ip> --cacert pebble_ca_bundle.pem` loop (or continuous s_client) sampling every ~0.2 s; count failures.
3. **Force renewal** — apply `renewal_window_ratio 1` + reload (lever above).
4. **Observe** — poll the served leaf until `serial != serial_1` (new issuance), capture leaf #2 -> `leaf_2.pem`, record `serial_2`, `notBefore_2`. Corroborate with the Caddy JSON log obtained/renewed event (string UNCONFIRMED — grep the `tls` logger).
5. **Assert (consume `cert_analyzer.sh`, Q5):**
   - `serial_2 != serial_1` AND `notBefore_2 > notBefore_1` (a REAL re-issue, not a cache reload — gotcha #11).
   - `cert_chain_roots_in leaf_2.pem pebble_ca_bundle.pem` (new cert still roots in the Pebble run's CA).
   - `cert_not_expired leaf_2.pem` and `cert_san_matches leaf_2.pem proxy.test`.
   - availability probe: **0 failed requests** across the swap (zero-downtime).
6. **Evidence (§11.4.69):** persist `leaf_1.pem`, `leaf_2.pem`, both serial/date captures, the Caddy renewal log line, and the availability-probe counter under `qa-results/<run-id>/letsencrypt_rotation/`.
7. **Reset** — remove `renewal_window_ratio 1` (revert to default `0.3333`).

Determinism (§11.4.50): run the rotation N× (default 3), asserting identical PASS + identical assertion outcomes; Pebble's per-run CA regeneration means the CA bundle is re-fetched each iteration. **Anti-bluff:** the proof is the changed-serial + re-chained + still-valid + zero-downtime tuple with captured PEMs — never a log line alone, never `PEBBLE_VA_ALWAYS_VALID` (which would not exercise DNS-01 and would be a §11.4 PASS-bluff for this phase).

Sources verified 2026-07-01

- Pebble README — https://github.com/letsencrypt/pebble/blob/main/README.md
- Pebble repository — https://github.com/letsencrypt/pebble

- Pebble test/certs README — https://github.com/letsencrypt/pebble/blob/main/test/certs/README.md
- Pebble issue #152 (root certificate retrieval / per-launch regeneration) — https://github.com/letsencrypt/pebble/issues/152
- Let's Encrypt blog — "How Pebble Supports ACME Client Developers" (2025-04-30) — https://letsencrypt.org/2025/04/30/pebbleacmeimplementation
- pebble-challtestsrv cmd README — https://github.com/letsencrypt/pebble/blob/main/cmd/pebble-challtestsrv/README.md
- challtestsrv repository — https://github.com/letsencrypt/challtestsrv
- challtestsrv README — https://github.com/letsencrypt/challtestsrv/blob/master/README.md
- pebble-challtestsrv Ubuntu manpage — https://manpages.ubuntu.com/manpages/noble/man1/pebble-challtestsrv.1.html
- Caddy `tls` directive — https://caddyserver.com/docs/caddyfile/directives/tls
- Caddy global options — https://caddyserver.com/docs/caddyfile/options
- Caddy automatic-https — https://caddyserver.com/docs/automatic-https
- Caddy `acme_server` directive — https://caddyserver.com/docs/caddyfile/directives/acme_server
- xcaddy — https://github.com/caddyserver/xcaddy
- caddy-dns/acmedns — https://github.com/caddy-dns/acmedns
- "How to use DNS provider modules in Caddy 2" — https://caddy.community/t/how-to-use-dns-provider-modules-in-caddy-2/8148
- certmagic (DNS01Solver / libdns) — https://pkg.go.dev/github.com/caddyserver/certmagic and https://github.com/caddyserver/certmagic
- caddy-dns/cloudflare (xcaddy Dockerfile pattern) — https://github.com/CaddyBuilds/caddy-cloudflare
- Caddy community — "How to force renewal of Let's Encrypt certificates" (Matt Holt: ratio=1 + reload) — https://caddy.community/t/how-to-force-renewal-of-lets-encrypt-certificates/14843
- Caddy issue #5589 (cert cache flush on config reload) — https://github.com/caddyserver/caddy/issues/5589
- DeepWiki — Caddy certificate storage & caching (renewAt formula) — https://deepwiki.com/caddyserver/caddy/4.2.3-certificate-storage-and-caching
- LE community — "Trouble setting up pebble-challtestsrv in local docker env" (IPv6 gotcha, -dnsserver wiring) — https://community.letsencrypt.org/t/trouble-setting-up-pebble-challtestsrv-in-local-docker-env/135870
- LE community — "SSL cannot get local issuer certificate when querying Pebble" — https://community.letsencrypt.org/t/ssl-

cannot-get-local-issuer-certificate-when-querying-pebble-server/230842
- Xoxzo blog — "Root certificates generation using ACME server Pebble" — https://blog.xoxzo.com/2020/11/18/root-certificates-generation-using-acme-server-pebble/

Honest gaps (UNCONFIRMED — method to obtain stated inline): exact Pebble-issued leaf validity period; whether Pebble honors a client-requested `notAfter`/`cert_lifetime`; whether this Pebble build serves ACME ARI; the exact Caddy "certificate obtained/renewed" log string. None block Phase 3 or Phase 5 — the force-renew lever and the changed-serial assertion are independent of all four.

Addendum 2026-07-01 — CoreDNS SOA-front (design-gap fix)

Status: Phase-3 hermetic issuance is GREEN — a real certificate was issued and verified (`cert_chain_roots_in` PASS against the per-run Pebble issuance CA). Getting there required correcting a design gap in the Option-A architecture above that only surfaced at runtime.

The design gap the original Option A missed

The Option-A plan (a custom Caddy libdns provider that POSTs the challenge TXT to challtestsrv `:8055/set-txt`) is correct about how the TXT is *published* — but it silently assumed certmagic would present the TXT and stop there. It does not. **Before** presenting the DNS-01 TXT, certmagic runs a **zone-determination SOA walk**: to know which zone to write `_acme-challenge.hermetic.test` into, it queries for the `SOA` record, walking UP the name (`_acme-challenge.hermetic.test.` → `hermetic.test.` → `test.` → .) until a nameserver answers authoritatively for a zone.

challtestsrv is a *mock* responder — it answers `A/AAAA/TXT/CNAME/CAA`, but it answers **`NOTIMP`** (not-implemented) to `SOA` queries. So certmagic's SOA walk got `NOTIMP` at every level and failed with:

```
could not determine zone for domain "_acme-
challenge.hermetic.test": ... NOTIMP
```

This was verified directly with `dig`:

```
dig @<challtestsrv-ip> -p 8053 SOA hermetic.test.
;; ->>HEADER<<- opcode: QUERY, status: NOTIMP, ...
```

The TXT-publish path (Q2/Q3) was never the blocker; the **zone-determination path that runs first** was. Option A as written had no SOA authority in the hermetic net, so certmagic could never even get to presenting the TXT.

The fix — insert a CoreDNS authoritative SOA front for `hermetic.test`

Add a **CoreDNS** service that is authoritative for `hermetic.test`, sitting in front of challtestsrv:

- **CoreDNS answers the `SOA`** for `hermetic.test` via its `template` plugin (`template IN SOA hermetic.test`), so certmagic's zone-determination walk succeeds at the `hermetic.test.` level.
- **CoreDNS `fallthrough`s every other qtype** (crucially the dynamic `_acme-challenge.hermetic.test TXT` that the libdns provider POSTs live) by `forward`ing to challtestsrv — so the actual challenge TXT is still served by challtestsrv, unchanged.

Two non-obvious details were load-bearing (both cost a debug cycle):

1. **The SOA MUST be returned in the ANSWER section, with owner = the zone apex `hermetic.test.`.** certmagic only accepts the zone from an **ANSWER-section** SOA whose owner name is the queried zone. A first attempt returned the SOA in the **AUTHORITY** section (the conventional place for SOA on a negative/referral answer) — certmagic did **not** treat that as "this is the zone", so the walk *continued upward* to `test.`, where CoreDNS is not authoritative and returned **`REFUSED`**, and issuance failed again. Putting the SOA in the ANSWER section with owner `hermetic.test.` stopped the walk at the right level. The CoreDNS `template` plugin is configured to emit the SOA as an ANSWER-section record for the apex.

2. **CoreDNS `forward` needs an IP, not a name.** The `forward . <upstream>` directive does not name-resolve its upstream inside the hermetic net (there is no bootstrap resolver for it), so the Corefile must point at challtestsrv's **live pod IP**. Because that IP changes per boot (rootless podman assigns it dynamically), `phase3_hermetic_issue.sh` **rewrites `coredns/Corefile` with challtestsrv's current pod IP on each boot** before starting CoreDNS.

Resulting wiring (supersedes the Option-A diagram for the SOA path)

```
                          hermetic podman net
 caddy (custom xcaddy + challtestsrv libdns)
   ACME_RESOLVERS=coredns:53   ---- SOA walk ---->  coredns
(authoritative hermetic.test)
```

```
                                                    |  template
IN SOA hermetic.test  (ANSWER, owner=apex)
                                                    |
fallthrough / forward . <challtestsrv-IP:8053>
   libdns POST set-txt :8055 -------------------->  challtestsrv
                                                    |  DNS
:8053  (A/AAAA/TXT/CNAME/CAA; NOTIMP for SOA)
 pebble  (VA)  -- TXT _acme-challenge query -->
challtestsrv:8053  (direct; NO SOA walk)
```

Key routing facts:

- **Caddy** is pointed at CoreDNS for its own pre-flight resolution: `ACME_RESOLVERS=coredns:53` (name-resolved inside the net — CoreDNS is the resolver, so certmagic's SOA walk hits CoreDNS first).
- **Pebble's VA** still queries **challtestsrv:8053 directly** (`pebble … -dnsserver challtestsrv:8053`). Pebble's validation does a straight `TXT _acme-challenge.<domain>` lookup — it does **not** do certmagic's zone-determination SOA walk — so it needs no SOA front and talks to challtestsrv unchanged. Only the *Caddy/certmagic* side needed CoreDNS.
- **CoreDNS forwards the TXT** (and A/AAAA/CAA) to challtestsrv, so the single live TXT the libdns provider publishes is what both the Caddy self-check (via CoreDNS→challtestsrv) and Pebble's VA (direct) observe — one source of truth for the challenge value.

Result (evidence)

With CoreDNS inserted, `phase3_hermetic_issue.sh` issues a **real certificate**: certmagic's SOA walk resolves the zone at `hermetic.test.`, the libdns provider publishes the TXT to challtestsrv, Pebble's VA validates it, and the issued leaf **chains to the per-run Pebble issuance CA** — `cert_chain_roots_in` PASS against the runtime-fetched `/roots/0` + `/intermediates/0` bundle (Q5). This is the concrete, working instantiation of Option A; the CoreDNS SOA-front is the missing piece the original Q3 write-up did not anticipate, added here so the architecture section reflects what actually issues a cert.

Honest boundary (§11.4.6): the CoreDNS-front necessity is FACT — the `NOTIMP`-on-SOA behaviour of challtestsrv and the ANSWER-vs-AUTHORITY-section SOA distinction were both confirmed by `dig` and by the reproduced failure→success transition, not assumed. The ANSWER-section requirement is certmagic's observed zone-acceptance behaviour in this build; the exact certmagic code path that rejects an AUTHORITY-section SOA was not read from source (method to confirm: trace

certmagic's `dns01` zone-lookup, which uses the ANSWER-section SOA owner as the zone). The end-to-end result (real cert issued + `cert_chain_roots_in` PASS) is the captured proof the wiring works.

Sources verified 2026-07-01 (addendum)

- CoreDNS `template` plugin (SOA synthesis, ANSWER-section records, `fallthrough`) — https://coredns.io/plugins/template/
- CoreDNS `forward` plugin (upstream MUST be an IP; no self-bootstrap name resolution) — https://coredns.io/plugins/forward/
- pebble-challtestsrv cmd README (mock DNS answers A/AAAA/TXT/CNAME/CAA; no SOA) — https://github.com/letsencrypt/pebble/blob/main/cmd/pebble-challtestsrv/README.md
- certmagic DNS-01 zone determination via SOA lookup (libdns `dns01`) — https://pkg.go.dev/github.com/caddyserver/certmagic and https://github.com/caddyserver/certmagic
- Pebble `-dnsserver` (VA points at challtestsrv directly; no SOA walk) — https://github.com/letsencrypt/pebble/blob/main/README.md